
Course Title: Object Oriented Design (OOD) & Object-Oriented Programming

Module Title: System Analysis, Design & Implementation

Assignment Type: UML Modelling

Project Title: ATM Machine

Method of Assessment: Oral Presentation

Nature: Group Work (Max 5 per group)

Project Date: 14-01-2025

Due Presentation Date: First lectures after Christmas break

GAME: Cave Run

Learning Outcomes

Part A: OOD

1. Completion of a game will solidify your understanding of object-oriented design
2. Sharpen your skills in analysis and design
3. Learning to overcome large software project complexities
4. Learning how to produce project artifacts
5. More...

Part B: OOP

1. Practice developing code incrementally – one class at a time
2. Testing your work along the way

Assignment overview

In the following, you will find your programming project requirement for this subject. You are strongly reminded to be honest. Any plagiarism is not tolerated. You should start working on your project now. Be reminded that a project not only consist of code, but must also consist of a corresponding documentation and explanation of your code.

As such, CC will help create the groups. Each group must consist of five (5) members maximum. Finally write a good, detailed report and present your work. The programming language for this project has to be C++.

Assignment division

This project will have two parts: **Part 1:** Analysis and Design in UML & **Part 2:** Implementation using C++ program

PART 1: OBJECT ORIENTED DESIGN

Your system should be described using the following UML diagrams.

1. **Use case Diagram:** Describe your system in terms of its main functions and the types/roles of users. Identify a few broad requirements of this system, and the way different types of users interact with it, using a use-case diagram.
2. **Class Diagram:** Describe your system involving a collaboration of classes. Use aggregation, generalization, association relationships and multiplicity numbers to create a class diagram for your system.
3. **Sequence Diagram:** Describe a particular scenario (actions that take place during a specific interval of time) in the working of a system, where the objects interact to perform some task. Draw the sequence diagram showing the time ordering of the object's interactions. Draw a sequence diagram for each "important" use case in the use case diagram.
4. **Collaboration Diagram:** Represent the above scenario in (3) as an equivalent collaboration diagram.
5. **State Diagram:** Draw a state diagram for each "important" object, where an object can have changes in its state depending on certain events and guard conditions.

Case Study Game Play

The objective of the **player** (marked "P" in **figure 1** below) is to escape from a cave that contains several rooms. Somewhere within the cave, however, is a **malicious** monster (marked "M") who will attempt to impede the player's progress! The player will begin in a room located in the bottom-left corner of the cave and will have to move through the rooms (while avoiding the monster) to reach the **exit**, which is at the top-right corner of the cave.

The player has a finite number of **health points**, which may decrease along the player's journey to the exit depending on the kinds of rooms they enter. **Normal** rooms do not cause the player to lose health points. **Poisonous** rooms (shown in green below) cause the player to lose health points after each move until the player has been "cured" (which happens with a certain probability or include some rooms with bonus health pickups similar to those discussed in the course game). There are

also **trap** rooms (shown in dark red) that cause the player's health points to decrease by some fixed amount and immediately end the player's turn. The player's quest will end abruptly if the monster catches them by moving into the same room, or if their health points are fully depleted.

The player makes up to **two moves** on each turn by clicking while the **monster** moves just once. If the player enters a poisonous or trap room, the room will change color so that you, the player, will know to avoid it from then on!

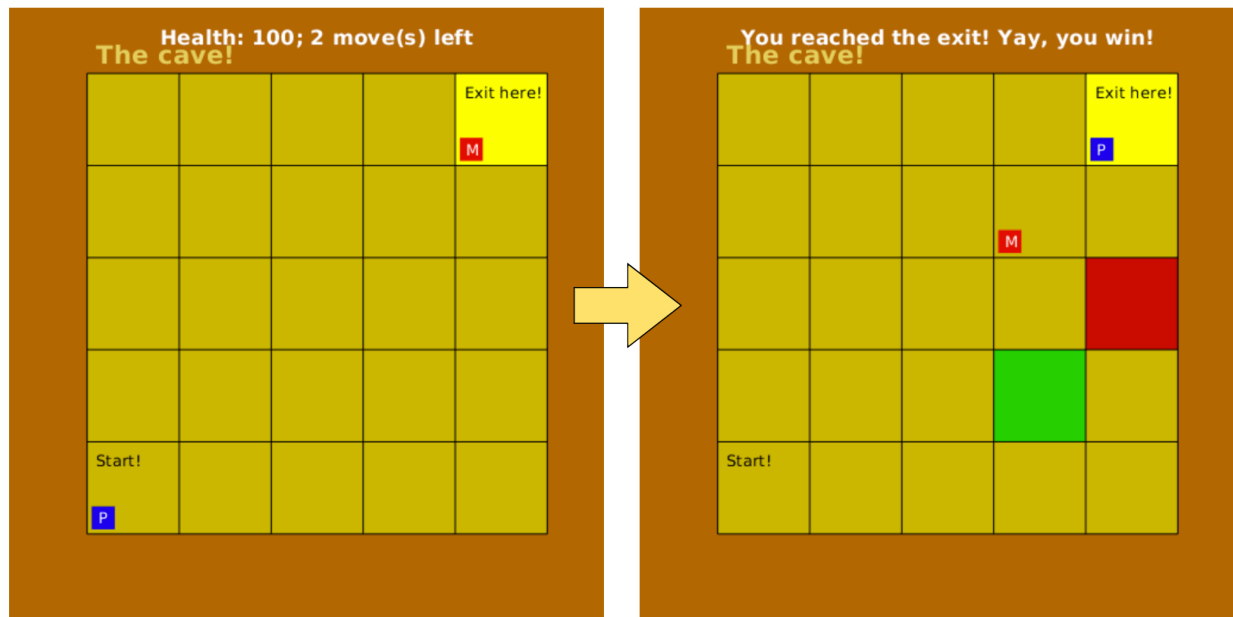


Figure 1: Game interface: The left image shows what the game looks like at the beginning, and the right image shows the board after the player has moved to the exit and won!

There are a few nouns used broadly in the game description, including **player**, **monster**, and **room**. These are the different entities that interact with each other in the game, and can therefore be thought of as classes in your design. Some nouns can be organized into broader categories; for example, while a **player** and a **monster** are different, they are still both **characters** in our game and share a few characteristics. So, you may design a **Character** class and then have two subclasses **Player** and **Monster**. Similarly, rooms with poison or traps are special kinds of rooms and can therefore be modeled as subclasses of a **Room** class.

These classes, and their public methods, form the **application programming interface**, or **API**, for this game. The class diagram serves as a “**road map**” for the capabilities of the objects that will make up the program. Knowing what features are and aren't available helps to guide your problem-solving process. Note that this diagram only shows one kind of relationship among objects: inheritance (“is-a”). You can draw additional diagrams for other kinds of relationships, such as composition (“has-a”). A Map has-an array of Rooms, for example.

By leveraging modular design, most of the **complexity** has been delegated to other classes, who manage their own data. The game itself just needs to check the game-over conditions and change whose turn it is (player or monster). When invoking characters' actions, the game is completely agnostic about whether the active character *c* is a player or a monster. In fact, the character actions of sense, move, and update are the same three steps that are used by both.

Note Better: The inspiration of the movements in this game comes from robot simulators: read sensor data, control actuators, and evolve the system and its environment forward in time.

Once you have a reasonable understanding of how the classes are intended to work, it's time to start implementing the game. Which class should you work on first? Look for one that is mostly independent of other unimplemented classes. For example, Player only depends generically on Room, but the subclasses of Room depend specifically on Player; therefore, Player is a good place to start.

Some Proposed Classes:

Characters

The **Character** class is an abstract base class—you will only ever construct subclasses of Character (like **Player** or **Monster**), never a generic Character itself. Nevertheless, it provides a lot of useful functionality.

A **Character** knows how to draw itself with a colored box and a small text label (one letter). It knows which room it is currently in, and it knows how to move between rooms. It also tracks how many moves it can still make in its current turn, and it has the ability to reset that allowance at the start of a new turn.

When moving, the Character class defers to its derived class to decide what direction to move in. It assumes that its derived class will implement a method that returns a vector of length 2 indicating how far to move horizontally and vertically – a return value of [0,1] means “move 0 columns to the right and 1 row up.” Most characters, including Player and Monster, can only move between adjacent rooms (diagonals included), so the elements returned by method are any combination of -1, 0, and 1.

In addition, subclasses can customize the behavior of three Character actions: **sense()**, **move()**, and **update()**. The **sense()** method accepts a Game as an input argument, allowing the character to query the overall game state via its public methods. **move()** takes care of moving the character to a new room; usually the base class behavior is sufficient, but it may be overridden if additional action is required as soon as a room is entered. **update()** is used to specify behavior that should occur due to the passage of time (rather than being triggered by movement to a location).

Player characters

A **Player** is a **Character** with the following tweaks:

- A **Player** should be shown graphically as a blue box containing the letter “P”.
- A **Player** keeps track of how many “health points” they have (initially 100).
- A **Player** may be damaged, depleting health points by a specified amount.
- A **Player** may be poisoned, in which case they lose a configured number of health points every time they **update()**.
- A **Player** has a configurable probability of curing their “poisoned” status every time they **update()**.
- A **Player** may move twice in a turn; this movement allowance should be replenished.
- A **Player** moves in the direction specified by the user when they click their mouse;
- A **Player** visits every room that they move into.

Most of the logic required to implement this behavior is trivial—often just one or two lines of code. Note that visiting rooms requires overriding the **move()** method inherited from class **Character**; however, we still want to do everything that the inherited **move()** method.

Observe that the player never needs to know what kind of **Room** they are visiting—the effect of visiting a room is delegated to the room itself, leveraging **polymorphism** to run the appropriate code for the type of room the player is in.

Monster characters

A **Monster** is a **Character** with the following tweaks:

- A **Monster** should be shown graphically as a red box containing the letter “M”.
- A **Monster** can sense where the player is on the map in **sense()**.
- When deciding where to move, a **Monster** looks at how close it is to the exit room and compares that with how close the player is to the exit. If the monster is closer to the exit than the player, then it will move to attack, advancing towards the player by one space. Otherwise, it moves towards the exit by one space.

Rooms

A **Room** is simply a location that a **Character** can enter. It has a position within a rectangular grid, specified by a column and row number (integers ≥ 1 , [x,y] ordering, positive y goes up—think Cartesian plots, not matrices) stored in a vector. When a player (specifically a **Player**, not a general **Character**) visits the room, the room may perform some action on the player in its **visit()** method; a generic room does not perform any action when visited, but subclasses of **Room** may.

Rooms are drawn as colored squares. The color may be changed at any time, and a text label may also be added to a room.

Poison Rooms

A **PoisonRoom** is a **Room** that poisons any **Player** who visits it. Once it has been visited, the player knows that the room is hazardous, so its color should change to green (you can print out a message in the console in our case without a GUI)

Trap Rooms

A **TrapRoom** is a **Room** that damages any **Player** who visits it by an amount specified when the room is constructed. Visiting such a room also **immediately ends** the player's turn by setting their number of remaining moves to zero. Once it has been visited, the player knows that the room is hazardous, so its color should change to red.

Map

A **Map** is a collection of rooms arranged in a rectangular grid. With a **Map**, it is easy to look up a room given its position in the grid (column and row numbers stored in a vector). A **Map** also knows the dimensions of the grid (which **Characters** can look up to avoid moving out-of-bounds), and it knows which position corresponds to the exit room.

Rooms can be added to a **Map** one at a time; since a **Room** knows its own position, the **Map** does not otherwise need to be told where to place it in the grid. Internally, Rooms are stored in a 2D cell array so that subclasses of **Room** can also be accommodated.

Game

Finally, a **Game** manages all of the objects involved in playing one round of the game; as mentioned before, it sits at the top of our **top-down design**. Its constructor's job is to set up the figure window for graphics, assemble the map, and create the two character objects. The **run/main()** method then alternates between these two characters, asking the "active" character to move until it runs out of moves, at which point its turn ends and the other character becomes "active". The **Game** is responsible for checking end-game conditions, since these usually involve conditions between multiple objects (are the player and monster in the same room? is the player in the exit room?). It also interfaces with the user by displaying useful information in the figure title.

Occasionally classes other than **Game** need a “global” view of the entire game state to do their work (for example, **Character.sense()**). A Game therefore has public methods to provide whatever information those other classes are allowed to see (for example, the player’s position).

When setting up the map, the game must decide which rooms should be hazardous (traps or poison). The start and exit rooms must never be hazardous.

Conclusion

That’s all that’s required for this project! But to appreciate the implications of object-oriented programming on the maintainability of software, it’s good to at least consider the work that would be required to add new features. Suppose, for example, that we wanted to add a “healing room” that restores health points when visited by a player. How much new code would you need to write? Or perhaps more importantly, how much existing code would not need to change? Adding a new subclass doesn’t require changing any other files except where objects of the new class are to be created. If you were working on a large team, this independence of modules might be a big benefit in terms of distributing work.

Brainstorm your own changes to this game that would be easy to implement in the context of its design. Could you have two players race for the exit? Could you have a computer-controlled player and a human-controlled monster? If you have a fun idea, share it on the class discussion board! Implementing such changes is a good way to practice and exercise your object-oriented programming skills. **Best of luck dear colleagues!**

PART 2: OBJECT ORIENTED PROGRAMMING - OOP

Your software design should be implemented in C++ code and show the following aspects of OOP and OOAD.

1. You should implement the concept of inheritance, polymorphism and other OOAD concept that you learn in these courses as necessary. You will find plenty of opportunities to apply OO concepts into your design and implementation.
2. You should spend time to design a usable, intuitive, and interface.
3. You should handle, if possible, all errors with meaningful error messages. The point here is to terminate gracefully and properly – you want to keep the users informed and aware of the state of the system.
4. Don’t copy any codes from other groups. Questions will be asked on coding during the presentation. In addition, the instructor will use anti-plagiarize internet-enabled software to

OOAD & OOP Course Group Assignment

automatically detect irregularities. If you or your group is caught plagiarizing, every member in your group will receive zero.

5. You are strongly encouraged to participate in all aspects of the development of this project not excluding problem solving, coding and documentation.